Ministry of Communications
and Information Technology

Digital Egypt Pioneers Builders
(Rwaad Misr)

Project Documentation

Wazuh SIEM & Windows Endpoint Monitoring

Persistence Threat Hunting & Automated SOC Alerting

A Centralized SIEM/XDR Environment with Internal Endpoint Threat Simulation and Discord Webhook Automation

Team 2 Members

Name	Role
Ahmed Ossama	Lead Technical Architect & Engineer
Mohamed Deyaa	Project Manager & Documentation Lead
Ali Mohamed	Detection & Telemetry Specialist
Hoda Alaa	System Integration Specialist
Eman Ayman	Penetration Tester & Threat Analyst
Mariam Salama	Quality Assurance & Technical Writer

Year: 2026

Submitted in fulfillment of the Digital Egypt Pioneers Builders Program requirements

Contents

List of Figures

List of Tables

Chapter 1

Project Planning & Management

1.1 Project Proposal

1.1.1 Overview

This project focuses on the architecture, configuration, and validation of a comprehensive Security Operations Center (SOC) environment. The core of the system utilizes the Wazuh Security Information and Event Management (SIEM) and extended Detection and Response (XDR) platform. The monitored infrastructure is deployed on a custom, isolated VMware NAT virtual switch (`VMnet8`) on the subnet `192.168.135.0/24`, and comprises a monitored Windows 10 endpoint (`DESKTOP-SLN103I`) instrumented with Microsoft Sysmon and a centralized Wazuh Manager. The threat simulation focuses on internal, host-based adversary behaviour — native background process creation and suspicious Dynamic Link Library (DLL) loads used for delayed-execution persistence. Detection events are surfaced in real time to a Discord channel that operates as a lightweight SOC incident dashboard.

1.1.2 Objectives

The primary goals of the project are:

- Achieve real-time telemetry ingestion from the Windows 10 endpoint (`DESKTOP-SLN103I`) into the centralized Wazuh Manager (`192.168.135.130`).
- Implement custom detection engineering mapped to the MITRE ATT&CK framework, including native process-creation detection (Wazuh Rule 67027) and suspicious DLL-load persistence detection (Wazuh Rule 92154 — `taskschd.dll`).
- **Deploy an automated alerting pipeline using a custom Python integration script (`custom-discord.py`)** that parses raw Wazuh JSON alerts, performs smart source-IP extraction, and formats structured payloads.
- **Operationalize a Discord Webhook as the SOC dashboard**, delivering dynamic, severity-based color-coded incident embeds (Green: Level 1–4, Orange: Level 5–9, Red: Level 10+).
- Conduct and validate threat-hunting simulations of internal malicious behaviour — background process spawning and `taskschd.dll` injection — on the Windows 10 endpoint.

1.1.3 Scope

The project scope covers deploying the Wazuh Manager, configuring the Windows 10 target endpoint with Microsoft Sysmon and File Integrity Monitoring (FIM), and instrumenting it for advanced endpoint telemetry. It further includes running an internal malware simulator on the endpoint to generate host-based persistence behaviour, developing custom XML detection rules, editing `ossec.conf` to register the `custom-discord.py` integration, and validating the end-to-end pipeline from the internal OS event to a color-coded Discord alert.

1.2 Project Plan

1.2.1 Timeline & Milestones (Gantt Chart)

The project follows a phased delivery model aligned with the official Rwaad Misr (Digital Egypt Pioneers Builders) submission deadlines. Figure 1.1 presents the visual Gantt chart, Table 1.1 lists the program milestone deadlines, and Table 1.2 decomposes each milestone into the detailed task schedule — including the Wazuh and Discord automation stack and the endpoint persistence / DLL-injection testing phase.

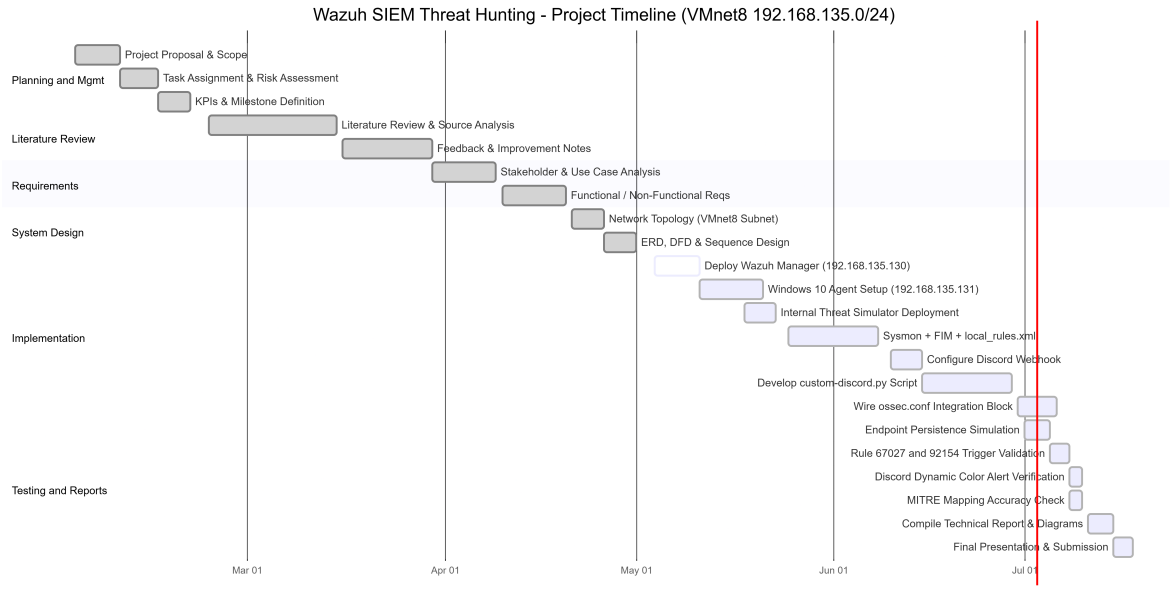


Figure 1.1: Project Gantt chart aligned to the Rwaad Misr milestone deadlines.

Table 1.1: Official Rwaad Misr Milestone Deadlines

Milestone	Student Deadline	Graduate Deadline
Project Planning & Management	20 Feb 2026	27 Jan 2026
Literature Review	20 Apr 2026	01 Mar 2026
Requirements Gathering	20 Apr 2026	01 Mar 2026
System Analysis & Design	01 May 2026	03 Apr 2026
Implementation (Source Code & Execution)	10 Jul 2026	17 May 2026
Final Presentation, Testing & Reports	17 Jul 2026	24 May 2026

Table 1.2: Project Timeline & Gantt Schedule (aligned to Student deadlines)

Task	Start	Dur.	Milestone (Deadline)
Project Proposal & Scope	02 Feb	7d	Planning & Mgmt (20 Feb)
Task Assignment & Risk Assessment	09 Feb	6d	Planning & Mgmt (20 Feb)
KPIs & Milestone Definition	15 Feb	5d	Planning & Mgmt (20 Feb)
Literature Review & Source Analysis	23 Feb	20d	Literature Review (20 Apr)
Feedback & Improvement Notes	16 Mar	14d	Literature Review (20 Apr)
Stakeholder & Use Case Analysis	30 Mar	10d	Requirements (20 Apr)
Functional / Non-Functional Reqs	10 Apr	10d	Requirements (20 Apr)
Network Topology (VMnet8 Subnet)	21 Apr	5d	System Design (01 May)
ERD, DFD & Sequence Design	26 Apr	5d	System Design (01 May)
Deploy Wazuh Manager (.130)	04 May	7d	Implementation (10 Jul)
Windows 10 Agent + Sysmon (.131)	11 May	10d	Implementation (10 Jul)
FIM + local_rules.xml (67027/92154)	25 May	14d	Implementation (10 Jul)
Configure Discord Webhook	10 Jun	5d	Implementation (10 Jul)
Develop custom-discord.py Script	15 Jun	14d	Implementation (10 Jul)

continued on next page

Table 1.2 – *continued from previous page*

Task	Start	Dur.	Milestone (Deadline)
Wire ossec.conf integration block	30 Jun	6d	Implementation (10 Jul)
Endpoint Persistence / DLL Sim	01 Jul	4d	Testing & Reports (17 Jul)
Rule 67027/92154 Validation	05 Jul	3d	Testing & Reports (17 Jul)
Discord Color Alert Verification	08 Jul	2d	Testing & Reports (17 Jul)
MITRE Mapping Accuracy Check	08 Jul	2d	Testing & Reports (17 Jul)
Compile Technical Report & Diagrams	11 Jul	4d	Testing & Reports (17 Jul)
Final Presentation & Submission	15 Jul	3d	Testing & Reports (17 Jul)

1.2.2 Deliverables

- Configured Wazuh SIEM/XDR platform with functional Windows 10 endpoint (Sysmon) telemetry.
- Custom MITRE-mapped detection rules in `local_rules.xml`.
- Automated response pipeline: `custom-discord.py` integration delivering color-coded Discord incident tickets.
- Validated threat-hunting evidence from the internal endpoint persistence / DLL-injection simulation.
- Complete technical documentation and diagrams hosted on GitHub.

1.2.3 Resource Allocation

Table 1.3: Resource Allocation

Resource	Purpose
VMware Workstation (VMnet8)	Isolated NAT virtual switch hosting the `192.168.135.0/24` lab subnet.
Wazuh Manager VM (Linux)	Central SIEM engine at `192.168.135.130`: log decoding, rule evaluation, alert generation.
Windows 10 Target VM (`DESKTOP-SLN103I`)	Monitored endpoint at `192.168.135.131` hosting the Wazuh Agent and Microsoft Sysmon for advanced process and DLL telemetry.
Internal Malware Simulator	Host-based tooling run locally on the Windows 10 endpoint to generate suspicious process creation and `taskschd.dll` load persistence behaviour.

Resource	Purpose
Python 3 (`requests`, `json`)	Runtime for the `custom-discord.py` integration script that parses alerts and formats embeds.
Discord Webhook (SOC Dashboard)	Outbound API endpoint receiving automated, severity-colored incident alerts as the SOC dashboard.
GitHub Repository	Version control for `local_rules.xml`, agent configurations, and documentation.

1.3 Task Assignment & Roles

Responsibilities are divided into two tracks: Technical Implementation and Project Documentation.

- **Ahmed Ossama — Lead Technical Architect & Engineer:** Owns the system architecture, Wazuh Manager deployment, Windows 10 endpoint (`DESKTOP-SLN103I`) configuration, and oversees custom detection engineering and endpoint threat simulations.
- **Mohamed Deyaa — Project Manager & Documentation Lead:** Owns the project plan, literature review, requirements formatting, and the final technical reports and presentation.
- **Ali Mohamed — Detection & Telemetry Specialist:** Customizes Windows Wazuh Agent telemetry, integrates Sysmon, and authors the MITRE-mapped rules in `local_rules.xml`.
- **Hoda Alaa — System Integration Specialist:** Configures the Sysmon integration and validates the `<syscheck>` FIM module for critical Windows directories and registry keys.
- **Eman Ayman — Penetration Tester & Threat Analyst:** Executes the internal malware simulation (suspicious process creation and `taskschd.dll` injection) on the endpoint to trigger the custom SOC alerts.
- **Mariam Salama — Quality Assurance & Technical Writer:** Records simulation metrics, drafts the user manual, and ensures GitHub documentation is well structured.

1.4 Risk Assessment & Mitigation Plan

Table 1.4: Risk Assessment & Mitigation

Risk	Mitigation
Malformed XML rules crashing the Wazuh Manager due to characters dropped when pasting large blocks into the terminal.	Use a "Here Document" with the `tee` command to bypass terminal buffer limits, and keep a crash-proof master copy of `local_rules.xml`.
Raw, noisy event logs overwhelming the SIEM analysis engine.	Customize the endpoint agent to monitor specific channels (e.g., PowerShell/Operational) and filter for specific Event IDs (4624, 4688, 4702).
Failure to detect host-based persistence mechanisms (e.g., scheduled-task DLL loads).	Configure Sysmon image-load monitoring and custom rules for suspicious modules such as `taskschd.dll` (Rule 92154).
Discord Webhook exposure or delivery failure breaking the alert pipeline.	Store the webhook URL only in `ossec.conf`, keep it out of version control, and validate delivery with a 204 response check inside `custom-discord.py`.

1.5 KPIs (Key Performance Indicators)

- **Detection Validation Metric:** A high volume of direct hits recorded on the Threat Hunting dashboard during simulation confirms accurate rule parsing (target: 420 direct hits).
- **Automation Trigger Accuracy:** Reliable execution of the `custom-discord.py` integration on every qualifying alert, with correct severity-to-color mapping delivered to the Discord SOC channel.
- **Threat Intelligence Correlation:** Successful mapping of triggered alerts to the correct MITRE ATT&CK techniques (e.g., T1053.005 for scheduled-task persistence, T1059 for process execution).

Chapter 2

Literature Review

This chapter records the academic evaluation of the project in line with the official Rwaad Misr (Digital Egypt Pioneers Builders) documentation guidelines. It captures the lecturer's feedback, suggested improvements, and the final grading criteria applied to the deliverables.

2.1 Feedback & Evaluation

The supervising lecturer's assessment of the SOC implementation and threat-hunting project is summarized below:

- **Strengths:** The project demonstrates a complete, working detection pipeline — from Sysmon endpoint telemetry on the Windows 10 host, through custom MITRE-mapped rule correlation on the Wazuh Manager, to an automated Discord notification channel. The addition of the `custom-discord.py` integration and the internal endpoint persistence simulation shows practical, end-to-end SOC engineering rather than a purely theoretical design.
- **Areas Observed:** Detection fidelity was validated against a live internal-threat simulation, with alerts correctly triggered (Rules 67027 and 92154) and forwarded as color-coded incident embeds. The isolated VMnet8 (`192.168.135.0/24`) topology keeps the exercise safe and reproducible.
- **General Comment:** The documentation is well structured and mapped to recognized frameworks (MITRE ATT&CK), and the automation component adds clear operational value for a SOC dashboard workflow.

2.2 Suggested Improvements

- Extend the detection coverage beyond process creation and DLL loads to include automated correlation rules for multi-stage persistence chains (e.g., a scheduled task created shortly after a suspicious process spawn).
- Add rate limiting and retry/back-off handling in `custom-discord.py` to protect against Discord API throttling during high-volume alert storms.

- Introduce threat-intelligence enrichment (e.g., VirusTotal or an IP reputation lookup) before an alert is posted to the SOC channel.
- Externalize the webhook URL into a protected secret store rather than `ossec.conf` to further harden the integration.

2.3 Final Grading Criteria

The final grade is distributed across documentation quality, technical implementation, testing/validation, and the final presentation, as detailed in Table 2.1.

Table 2.1: Final Grading Criteria

Criterion	**Weight**	**Focus Area**
Documentation Quality	25%	Structure, clarity, diagrams, guideline compliance
Technical Implementation	35%	Wazuh deployment, custom rules, `custom-discord.py` integration
Testing & Validation	25%	Endpoint persistence / DLL simulation, rule/alert accuracy, MITRE mapping
Final Presentation	15%	Delivery, live demonstration, Q&A handling
Total	**100%**	

Chapter 3

Requirements Gathering

This chapter identifies the stakeholders, articulates the user stories, and defines the functional and non-functional requirements that the SOC environment must satisfy. The requirements explicitly account for the automated Discord alerting pipeline and the resilience of the platform under active internal-threat simulation.

3.1 Stakeholder Analysis

Identifying the key users and stakeholders ensures the system is built to meet operational needs.

- **SOC Analysts & Incident Responders:** Require a centralized dashboard with real-time log ingestion and alerts mapped to MITRE ATT&CK. They additionally rely on the Discord SOC channel for instant, at-a-glance, color-coded incident notifications away from the main console.
- **Detection Engineers:** Require the ability to write, test, and deploy custom XML rules without destabilizing the Manager, and to route qualifying alerts through the `custom-discord.py` integration.
- **Security Architects / System Administrators:** Require reliable deployment of the Wazuh Manager and the Windows 10 Agent, and secure handling of the outbound webhook integration.
- **Penetration Testers (Red Team):** Require a safe, isolated Windows 10 endpoint (`DESKTOP-SLN103I`, `192.168.135.131`) on which to run internal malware behaviour (process spawning, DLL injection) and validate that host-based actions reliably generate detections.

3.2 User Stories & Use Cases

3.2.1 User Stories

The following user stories capture the primary interactions between the security team and the SOC environment. Each is coded (`US`) and prioritized as *Must* or *Should*.

Table 3.1: User Stories

Code	User story	Must/ Should	Comments
US01	As a SOC Analyst, I want alerts mapped to the MITRE ATT&CK framework so that I can track and understand simulated internal threats.	Must	Threat hunting.
US02	As an Incident Responder, I want to be alerted when a suspicious background process is created (Rule 67027) so that I can identify malicious execution on the Windows endpoint.	Must	Endpoint security.
US03	As a Security Engineer, I want an instant alert when a process loads `taskschd.dll` (Rule 92154) so that I can detect delayed-execution persistence tasks.	Must	Persistence / DLL load.
US04	As an on-call SOC Analyst, I want endpoint persistence detections pushed to a Discord channel as a color-coded embed so that I am notified of high-severity incidents in real time.	Must	Discord automation.
US05	As a Detection Engineer, I want to deploy custom XML rules without destabilizing the Manager so that I can extend coverage safely.	Should	Rule stability.

3.2.2 Use-Case Scenarios

Table 3.2: Use Case UC-01 — Simulate Endpoint Persistence (DLL Injection)

Use Case Number:	UC-01	
Use Case Name:	Simulate Endpoint Persistence (DLL Injection)	
Actors:	Internal Malware Simulator (Local Threat)	
Overview:	The local threat spawns a suspicious background process that loads `taskschd.dll` to register a delayed-execution persistence task, generating Sysmon telemetry.	
Related Use Cases:	Detect & Alert (UC-02)	
Event (stimulus):	**User Actions:** 1. Spawn a suspicious process on `DESKTOP-SLN103I`. 2. Process loads `taskschd.dll`.	**System Action:** 1. Sysmon emits Event ID 1 (Process Create). 2. Sysmon emits Event ID 7 (Image Loaded) to the monitored channel.
Exceptions:	Sysmon channel not monitored, agent offline, event outside telemetry scope.	
Comments:	Primary internal-threat simulation used to validate the detection pipeline.	

Table 3.3: Use Case UC-02 — Detect & Alert via Discord

Use Case Number:	UC-02	
Use Case Name:	Detect & Alert via Discord	
Actors:	Wazuh Manager, `custom-discord.py`, SOC Analyst	
Overview:	The Manager correlates the Sysmon events, triggers a rule, and the Python integration posts a color-coded embed to Discord.	
Related Use Cases:	Simulate Endpoint Persistence (UC-01)	
Event (stimulus):	**User Actions:** 1. Agent forwards events to the Manager. 2. (Automated — no analyst input.)	**System Action:** 1. Decodes events and matches Rules 67027 / 92154. 2. Invokes `custom-discord.py`; source IP resolves to N/A. 3. Maps severity to color and POSTs the embed (endpoint `DESKTOP-SLN103I`).
Exceptions:	Webhook unreachable, Discord API rate limit (HTTP 429), malformed JSON alert.	
Comments:	Core automation use case; validates the end-to-end SOC alert flow.	

3.3 Functional Requirements

The functional requirements are coded (`FR`) and grouped by capability area, following the standard *Code / Requirements statement / Must-Should / Comments* format.

3.3.1 Log Ingestion & Endpoint Telemetry

Table 3.4: Functional Requirements — Log Ingestion & Telemetry

Code	Requirements statement	Must/ Should	Comments
FR001	The Wazuh Manager (`192.168.135.130`) must receive and analyze telemetry from the Windows 10 endpoint (`DESKTOP-SLN103I`).	Must	Central aggregation.
FR002	The Windows Wazuh Agent must integrate with Microsoft Sysmon to collect process-creation (Event ID 1) and image-load (Event ID 7) telemetry.	Must	Advanced telemetry.

Code	Requirements statement	Must/ Should	Comments
FR003	The `<syscheck>` module must perform real-time FIM on critical Windows directories and Registry keys (Run, RunOnce, Winlogon).	Must	Real-time FIM.

3.3.2 Detection & Rule Engine

Table 3.5: Functional Requirements — Detection & Rules

Code	Requirements statement	Must/ Should	Comments
FR004	The system must support custom detection engineering via `local_rules.xml`, including native process-creation detection (Rule 67027).	Must	Custom rule parsing.
FR005	The system must detect suspicious module loads, specifically a process loading `taskschd.dll` indicating a delayed-execution persistence task (Rule 92154).	Must	Persistence detection.
FR006	Triggered alerts must be mapped to the correct MITRE ATT&CK technique (e.g., T1053.005 scheduled-task persistence, T1059 process execution).	Must	Framework mapping.

3.3.3 Automated Discord Alerting

Table 3.6: Functional Requirements — Discord Automation

Code	Requirements statement	Must/ Should	Comments
FR007	On every qualifying alert the Manager must invoke the `custom-discord.py` integration registered in `ossec.conf`.	Must	Python webhook.
FR008	The script must perform smart source-IP extraction (searching `data.srcip` and `win.eventdata.ipAddress`) and gracefully assign `N/A` for internal OS events with no network source.	Must	Multi-source parsing.
FR009	The integration must set the embed border color by severity: Green (Level 1–4), Orange (Level 5–9), Red (Level 10+).	Must	Dynamic color coding.

Code	Requirements statement	Must/ Should	Comments
FR010	The script must post a structured embed (Rule ID, target endpoint `DESKTOP-SLN103I`, severity, source IP) to the Discord Webhook SOC channel.	Must	Alert delivery.

3.4 Non-Functional Requirements

The performance, security, and reliability criteria are coded (`NFR`) using the same format.

Table 3.7: Non-Functional Requirements

Code	Requirements statement	Must/ Should	Comments
NFR01	**Performance:** The Windows agent must collect only actionable threat data, filtering noisy raw event logs before transmission.	Must	Log filtering.
NFR02	**API Response Handling:** The `custom-discord.py` integration must validate the Discord response (expect `204 No Content`), fail safely without crashing the integration daemon, and avoid data loss when the endpoint is unreachable or rate-limited.	Must	Webhook resilience.
NFR03	**Stability Under Load:** The pipeline must sustain a rapid burst of endpoint events (process spawns and DLL loads) without dropping alerts, crash-looping, or blocking log processing.	Must	Event-burst resilience.
NFR04	**Reliability:** Custom rule deployment must use the secure "Here Document" (`tee`) approach to prevent malformed-XML crash loops.	Must	System stability.
NFR05	**Accuracy:** Detection rules must produce valid alerts with minimal false positives (target metric: 420 direct hits).	Must	Alert fidelity.
NFR06	**Security:** The Manager must securely handle sensitive telemetry, and the Discord webhook URL must be protected and kept out of version control.	Must	Data protection.
NFR07	**Usability:** Discord alerts must remain legible using redundant text labels alongside color so severity is clear without relying on color alone.	Should	Accessibility.

Chapter 4

System Analysis & Design

4.1 Problem Statement & Objectives

4.1.1 Problem Statement

Modern enterprise endpoints are a primary target for adversaries who rely on native operating-system mechanisms — background process creation and malicious DLL loads — to execute code and establish stealthy persistence, yet these host-based behaviours frequently go undetected without centralized endpoint visibility. In a small lab or SOC training context, analysts additionally lack an out-of-band, real-time notification channel: they must watch the dashboard continuously to notice a live incident. This project addresses both gaps by building a centralized SIEM/XDR pipeline on the isolated VMnet8 subnet (`192.168.135.0/24`) that monitors internal Windows 10 endpoint telemetry, and by pushing high-severity detections to a Discord channel that acts as a lightweight, always-on SOC dashboard.

4.1.2 Objectives

- Implement a centralized SIEM/XDR platform (Wazuh Manager) providing real-time telemetry ingestion from the Windows 10 endpoint (`DESKTOP-SLN103I`).
- Perform detection engineering mapped to the MITRE ATT&CK framework, including process-creation detection (Rule 67027) and `taskschd.dll` persistence detection (Rule 92154, T1053.005).
- Validate the pipeline with a live internal malware simulation — suspicious process spawning and DLL injection — on the endpoint.
- Automate incident notification through the `custom-discord.py` integration, delivering severity-based color-coded embeds to the Discord SOC channel.

4.1.3 Use Case Diagram

Figure 4.1 identifies the system actors — the Internal Threat (Local Malware Simulator), Detection Engineer, SOC Analyst, and the Discord SOC Channel — and their interactions with the SOC environment.

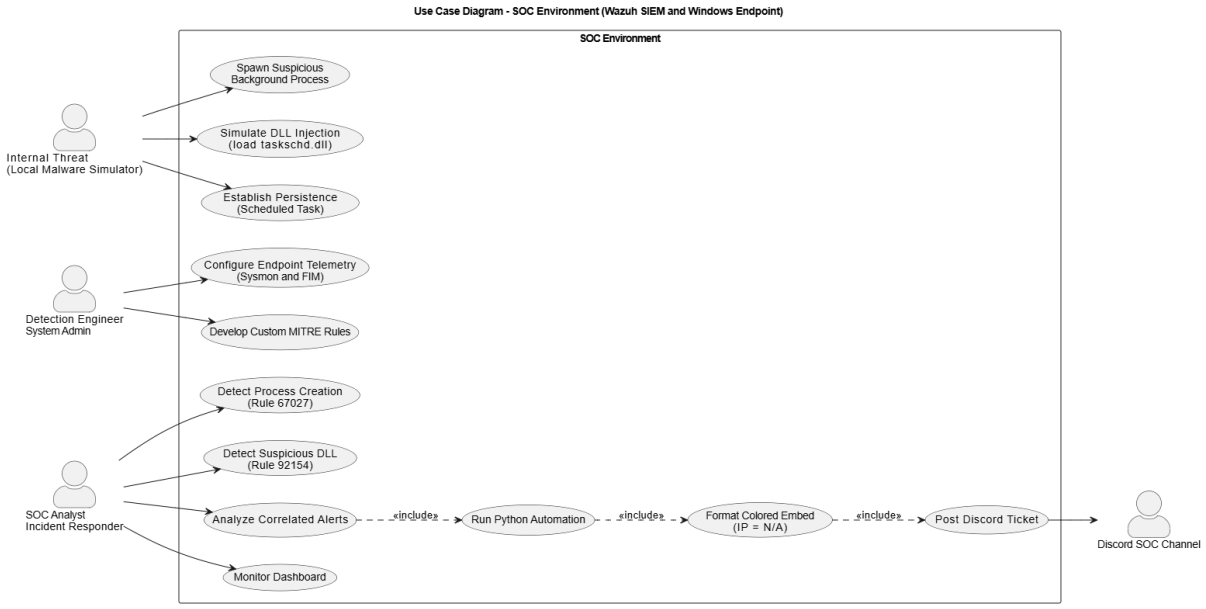


Figure 4.1: Use Case Diagram of the SOC environment (Wazuh SIEM & Windows Endpoint).

4.2 Software Architecture

The system implements a **Hub-and-Spoke (Centralized SIEM) architecture**. The Wazuh Manager acts as the central aggregation and analysis engine (the *Hub*), while the Wazuh Agent deployed on the Windows 10 endpoint acts as the telemetry collector (the *Spoke*). The spoke forwards filtered, encrypted Sysmon telemetry to the hub, which decodes it, evaluates it against `local_rules.xml`, and indexes the resulting alerts.

This project extends the classic Hub-and-Spoke model with an **outbound automation edge**. When a qualifying alert is generated, the Manager invokes the `custom-discord.py` integration script, which formats the alert and performs an outbound HTTPS call to the **Discord Webhook API**. The Discord channel therefore becomes an additional, cloud-hosted presentation layer alongside the on-premises Wazuh Dashboard — extending the hub's reach beyond the isolated lab network without exposing the internal infrastructure.

4.2.1 Database Design & Data Modeling

As a SIEM, Wazuh uses a NoSQL/document-based indexer rather than a traditional relational database. Data is stored as JSON documents in the Wazuh Indexer, with decoders parsing raw Sysmon events into structured key–value pairs (e.g., `rule.id`, `agent.name`, `win.eventdata.image`). Relationships are established through rule correlation: a `Security_Alert` entity is generated when a `Raw_Log_Event` matches a `Detection_Rule`, and each alert may in turn produce a `Discord_Ticket`. Figure 4.2 presents the logical entity-relationship model.

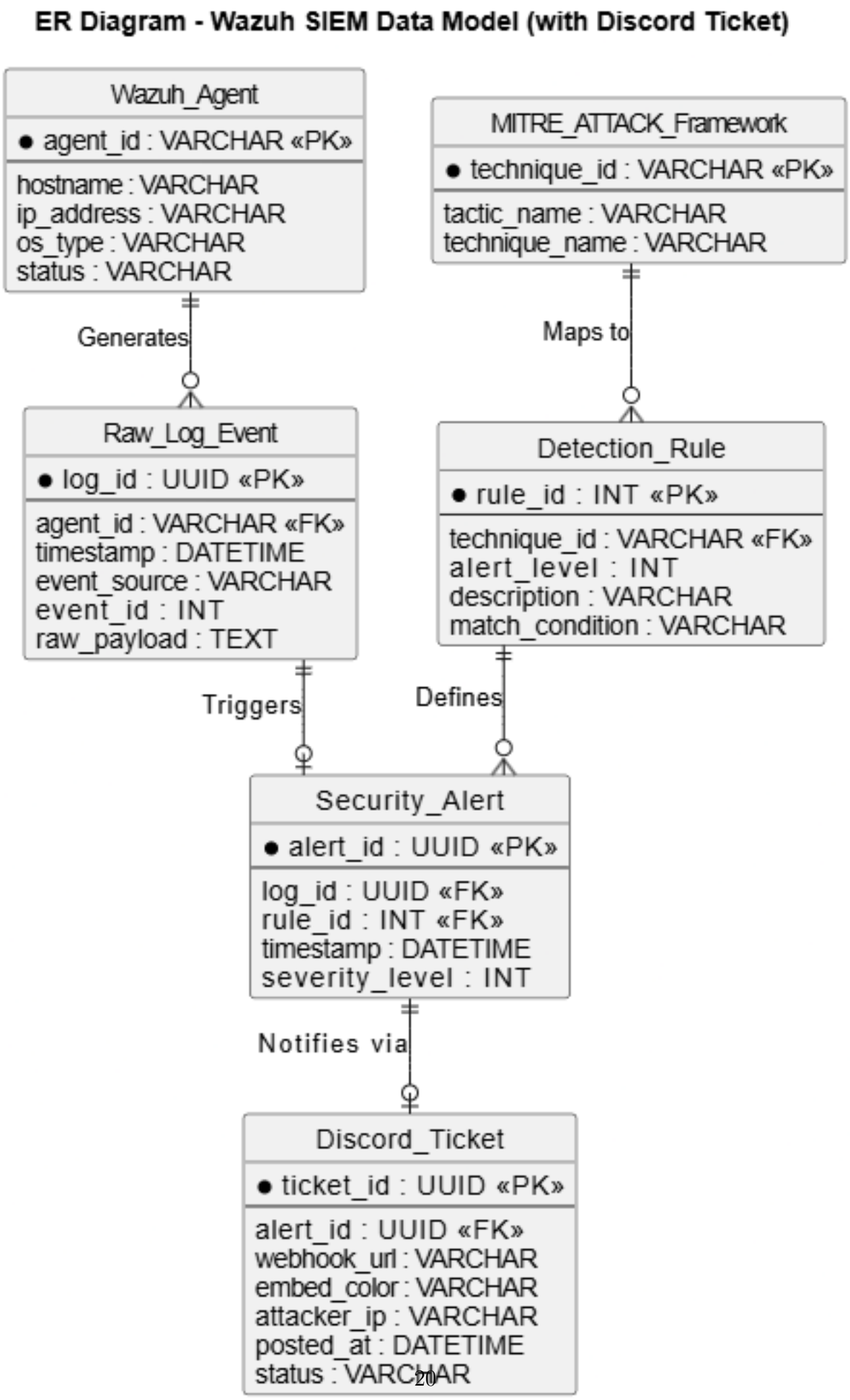


Figure 4.2: Entity-Relationship Diagram (including the Discord_Ticket entity).

4.3 Data Flow & System Behavior

The end-to-end detection pipeline flows as follows:

1. **Internal Threat (Local Malware Simulator):** On the Windows 10 endpoint (`DESKTOP-SLN103I`), the simulator spawns a suspicious background process that loads `taskschd.dll` to register a delayed-execution persistence task.
2. **Data Generation (Sysmon):** Sysmon captures the behaviour, emitting Event ID 1 (Process Create) and Event ID 7 (Image Loaded).
3. **Data Collection (Wazuh Agent):** The Agent reads and filters the event data and forwards it over an encrypted channel to the Manager.
4. **Data Processing (Wazuh Manager):** The Manager decodes the events, correlates them, and triggers Rules 67027 and 92154, mapping the persistence behaviour to MITRE T1053.005.
5. **Integration (`custom-discord.py`):** The Manager invokes the Python script, which resolves the source IP (`N/A` for this internal OS event), maps severity to a color, and formats a JSON embed.
6. **Response (Discord API):** The script POSTs the embed to the Discord Webhook, and a color-coded incident card for `DESKTOP-SLN103I` appears instantly in the SOC channel.

4.3.1 Data Flow Diagram (DFD)

Figure 4.3 shows the data flow from the internal malware simulator and Sysmon, through the Wazuh collection and analysis processes, to the indexer, dashboard, and the extended `custom-discord.py` → Discord API → SOC Analyst output path.

Figure 4.3: Data Flow Diagram of the SOC detection and alerting pipeline.

4.3.2 Sequence Diagram (Endpoint Detection Flow)

Figure 4.4 details the ordered message exchange, from the suspicious process execution and `taskschd.dll` load, through Sysmon capture and Manager rule processing, to the dynamic colored Discord embed.

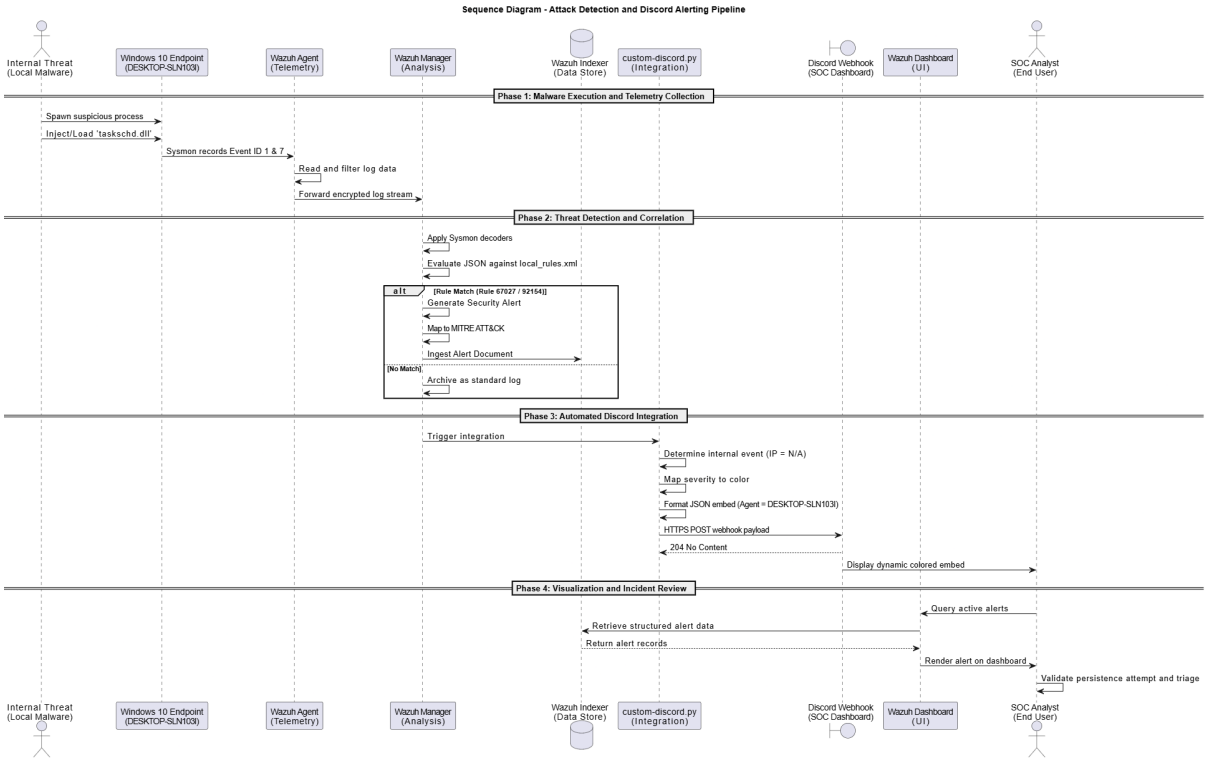


Figure 4.4: Sequence Diagram of the endpoint persistence detection and Discord alerting pipeline.

4.3.3 Activity Diagram

Figure 4.5 presents the swimlaned workflow across the Target Environment, Wazuh Agent, Wazuh Manager, the Discord/Python Integration, and the SOC Analyst.

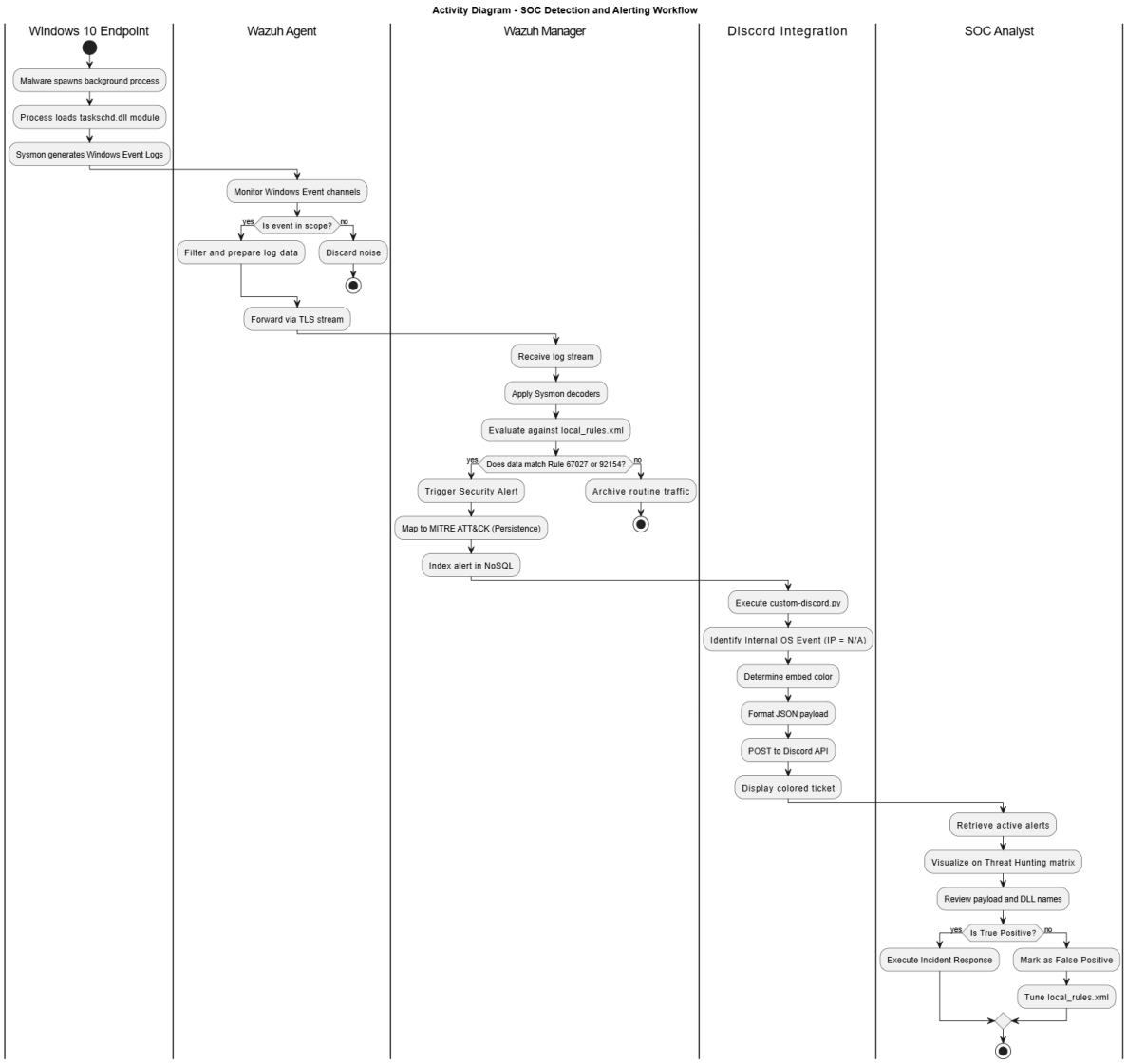


Figure 4.5: Activity Diagram of the SOC detection and alerting workflow.

4.3.4 State Diagram

Figure 4.6 models the Wazuh Agent lifecycle, transitioning from *Unregistered* through *WaitingForAuth* to the *Registered* composite state (Disconnected, Active, ErrorState).

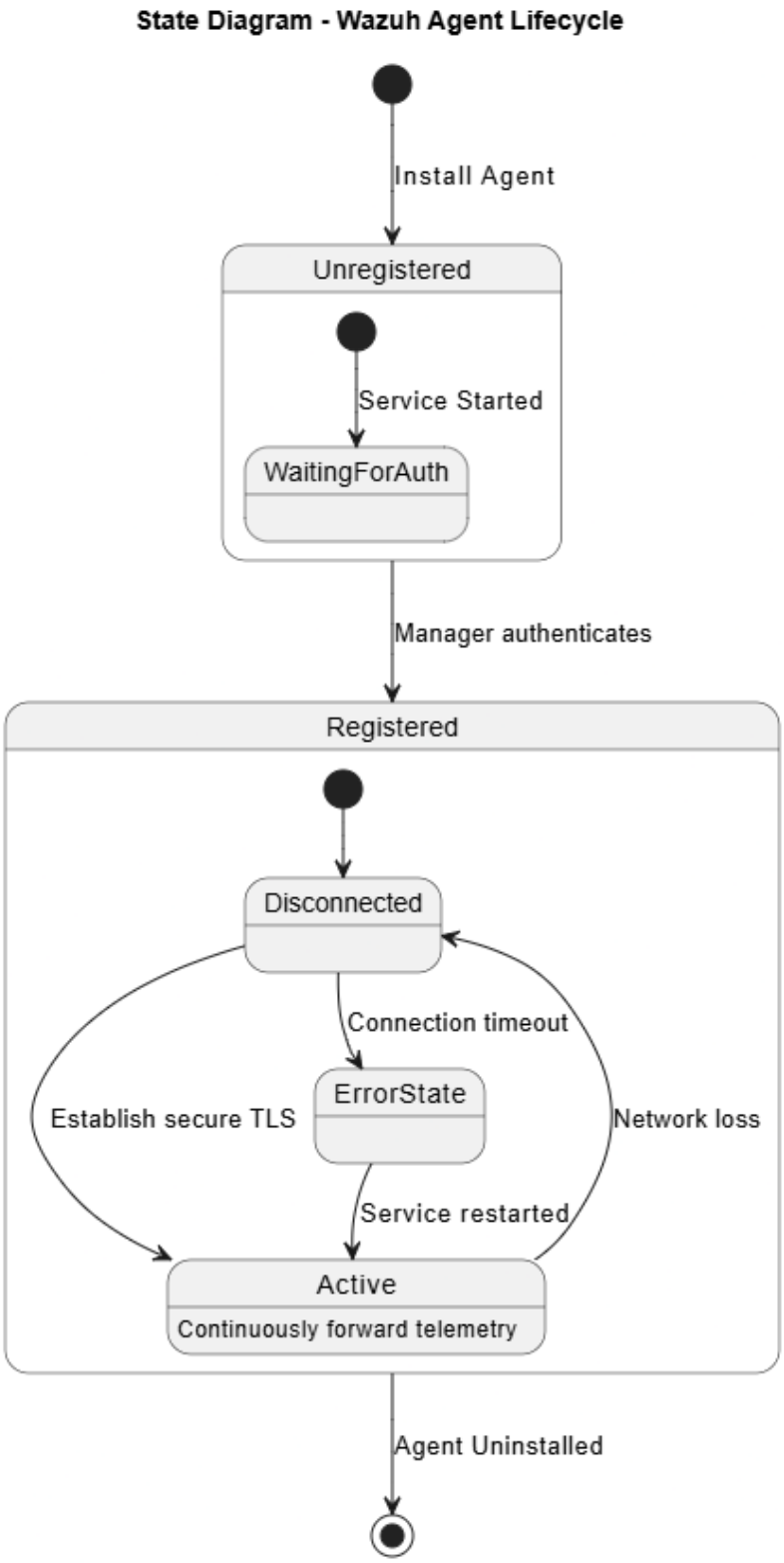


Figure 4.6: State Diagram of the Wazuh Agent lifecycle.

4.4 UI/UX Design

The system presents two complementary interfaces: the on-premises **Wazuh Dashboard** for deep analysis and threat hunting, and the cloud-based **Discord channel** for instant, at-a-glance incident notification.

4.4.1 Wazuh Dashboard

The Wazuh Dashboard (OpenSearch-based) is the primary SOC console. It follows a top-down, progressive-disclosure flow: a central overview with agent-summary and severity breakdown widgets, drill-down into tactical MITRE mapping, and finally granular "Document Details" views exposing the raw JSON payload. A strict semantic color system is used — Deep Red for critical (Level 15+), Bright Orange for high (Level 12–14), and Yellow/Blue for medium and low — so that the highest-severity incidents are immediately visible. Figure 4.7 shows the central SOC overview.

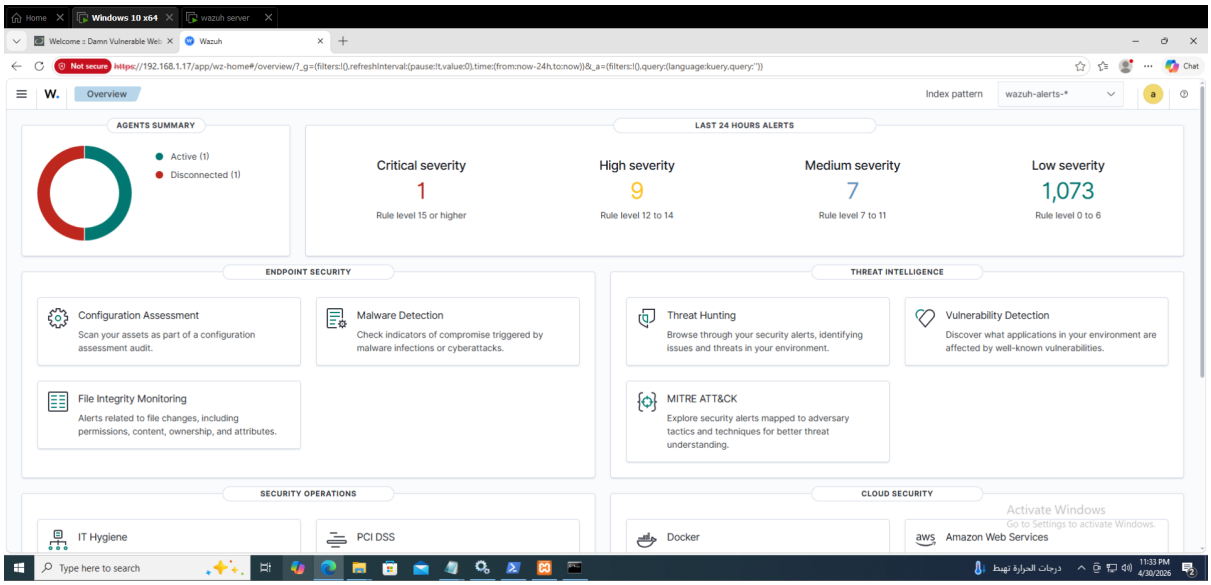


Figure 4.7: Wazuh central SOC overview dashboard.

The MITRE ATT&CK mapping matrix (Figure 4.8) provides immediate context on adversary tactics, mapping triggered alerts across tactics (e.g., Execution, Persistence) and listing techniques such as T1053.005 (Scheduled Task) and T1059 (Command & Scripting Interpreter) alongside their alert counts.

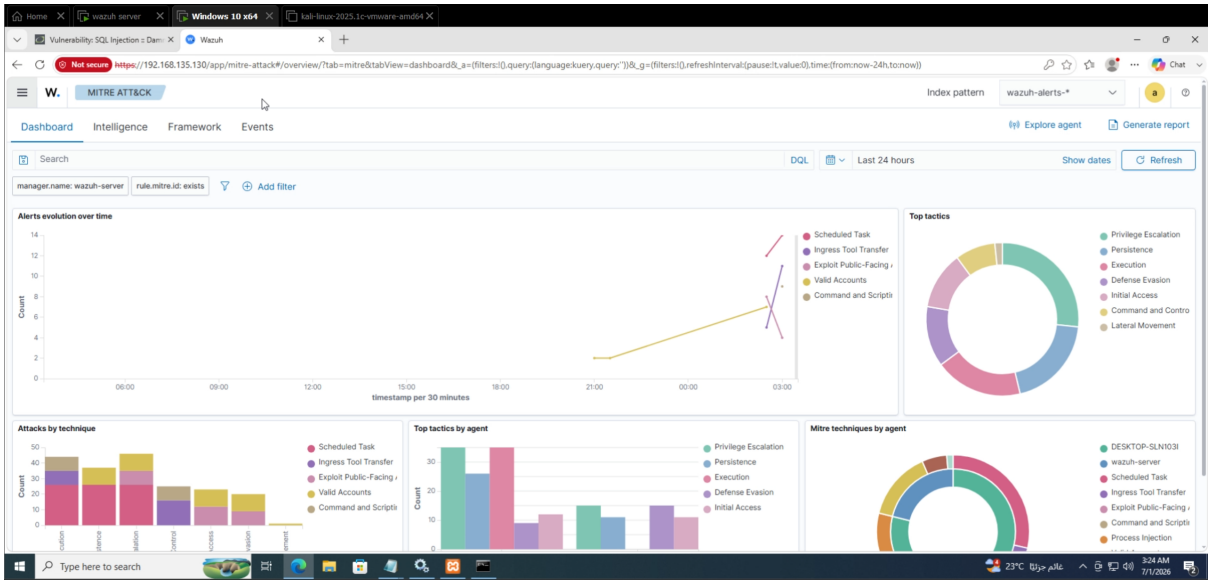


Figure 4.8: MITRE ATT&CK framework mapping matrix in the Wazuh Dashboard.

4.4.2 Discord SOC Interface

The Discord channel serves as a lightweight, always-on SOC dashboard. Each incident arrives as a graphical embed (card) whose border color is set dynamically by the `custom-discord.py` script according to the Wazuh severity level — Green (Level 1–4), Orange (Level 5–9), and Red (Level 10+). Every embed carries redundant text fields (Rule ID, target endpoint `DESKTOP-SLN103I`, severity, and source IP — shown as `N/A` for internal OS events) so severity is clear without relying on color alone. Figure 4.9 shows a color-coded alert as rendered in the Discord SOC channel.

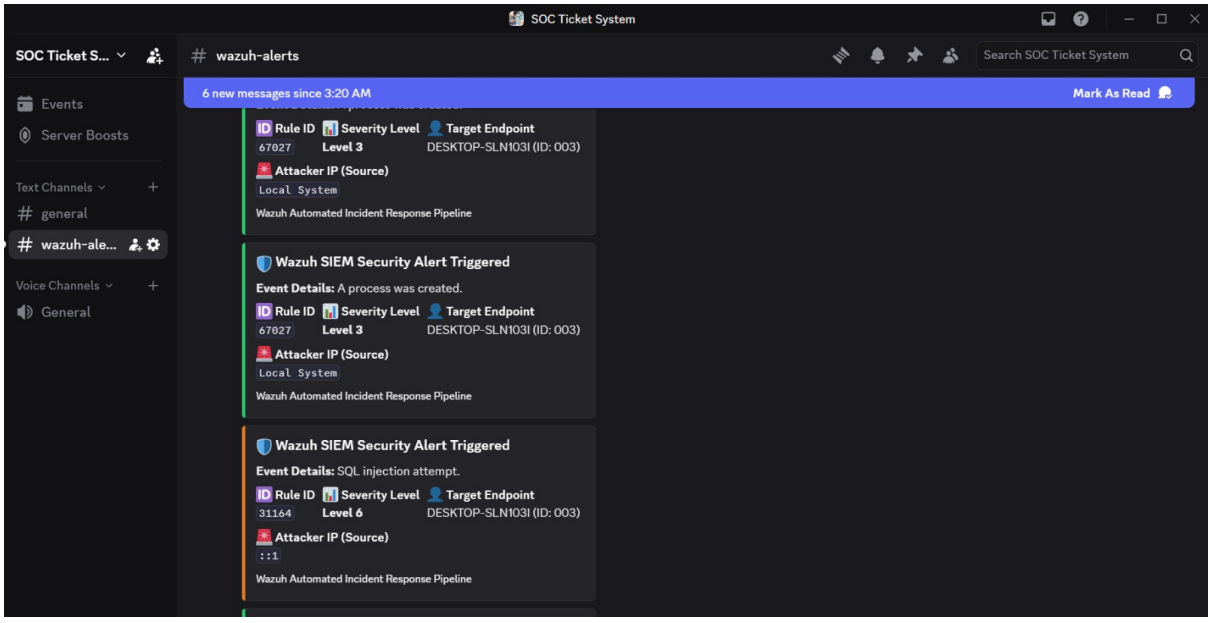


Figure 4.9: Color-coded incident embed in the Discord SOC channel.

4.5 System Deployment

The deployment is a distributed Virtual Machine (VM) model on VMware Workstation, simulating a localized enterprise network on the isolated NAT virtual switch `VMnet8` (subnet `192.168.135.0/24`). Table 4.1 defines the exact host addressing.

Table 4.1: Deployment Node Addressing (VMnet8 — `192.168.135.0/24`)

Node	**IP Address**	**Role**
Wazuh Manager (Linux)	`192.168.135.130`	Central SIEM hub: event decoding, rule evaluation, alert generation, and `custom-discord.py` integration.
Windows 10 Target (`DESKTOP-SLN103I`)	`192.168.135.131`	Monitored endpoint hosting the Wazuh Agent, Microsoft Sysmon, and the internal malware simulator.
Discord Webhook API	Outbound (HTTPS)	Cloud SOC dashboard receiving severity-colored incident embeds.

Figure 4.10 shows the full deployment topology, including the VMnet8 subnet, the Manager and Windows 10 endpoint VMs, and the outbound API edge to the Discord Webhook.

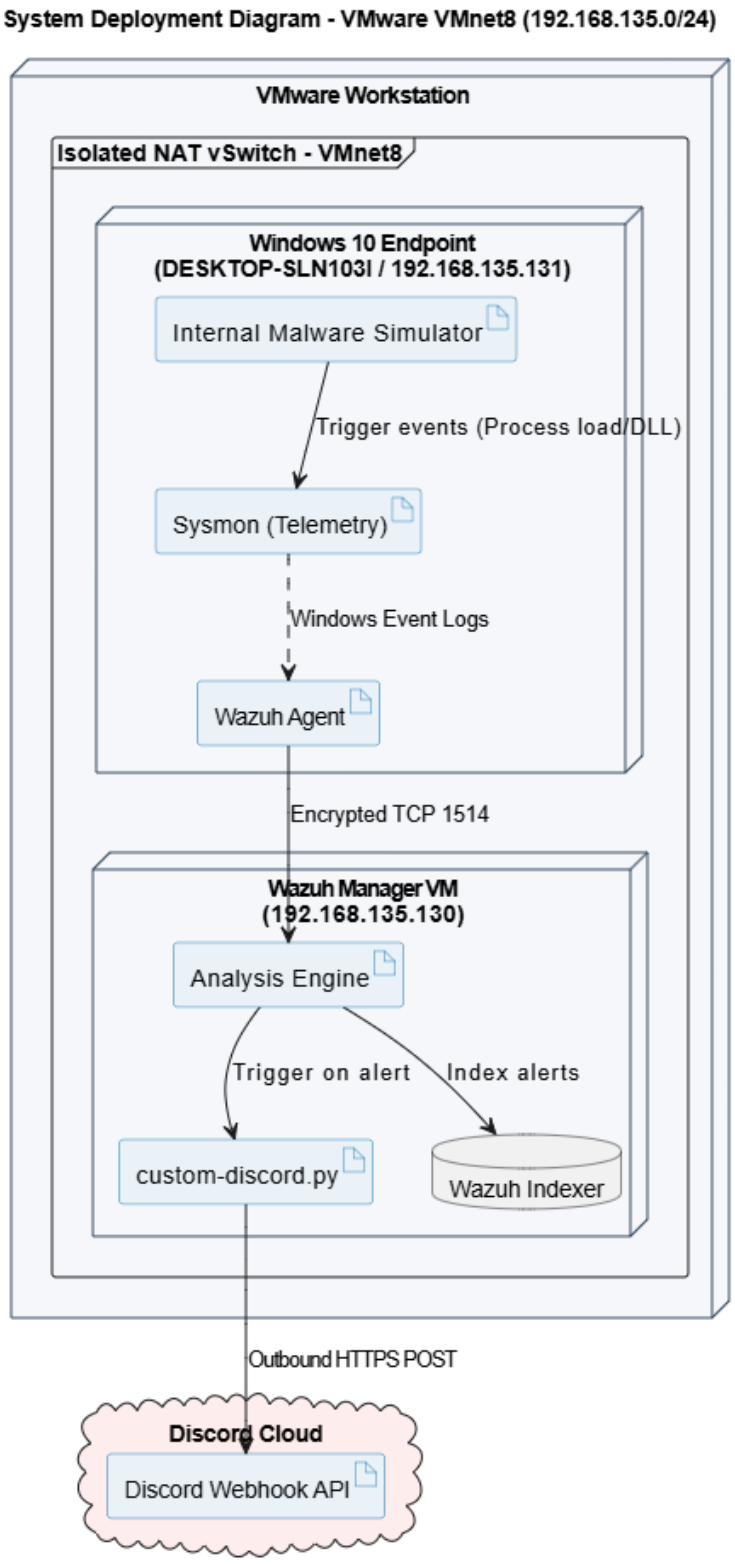


Figure 4.10: System Deployment Diagram (VMnet8 `192.168.135.0/24` with Discord API edge).

4.5.1 Component Diagram

Figure 4.11 shows the high-level software components and their dependencies — from the internal malware simulator and Sysmon, through the Wazuh Agent, decoders, and rule engine, to the `custom-discord.py` integration, the Discord Webhook API, and the Wazuh Dashboard.

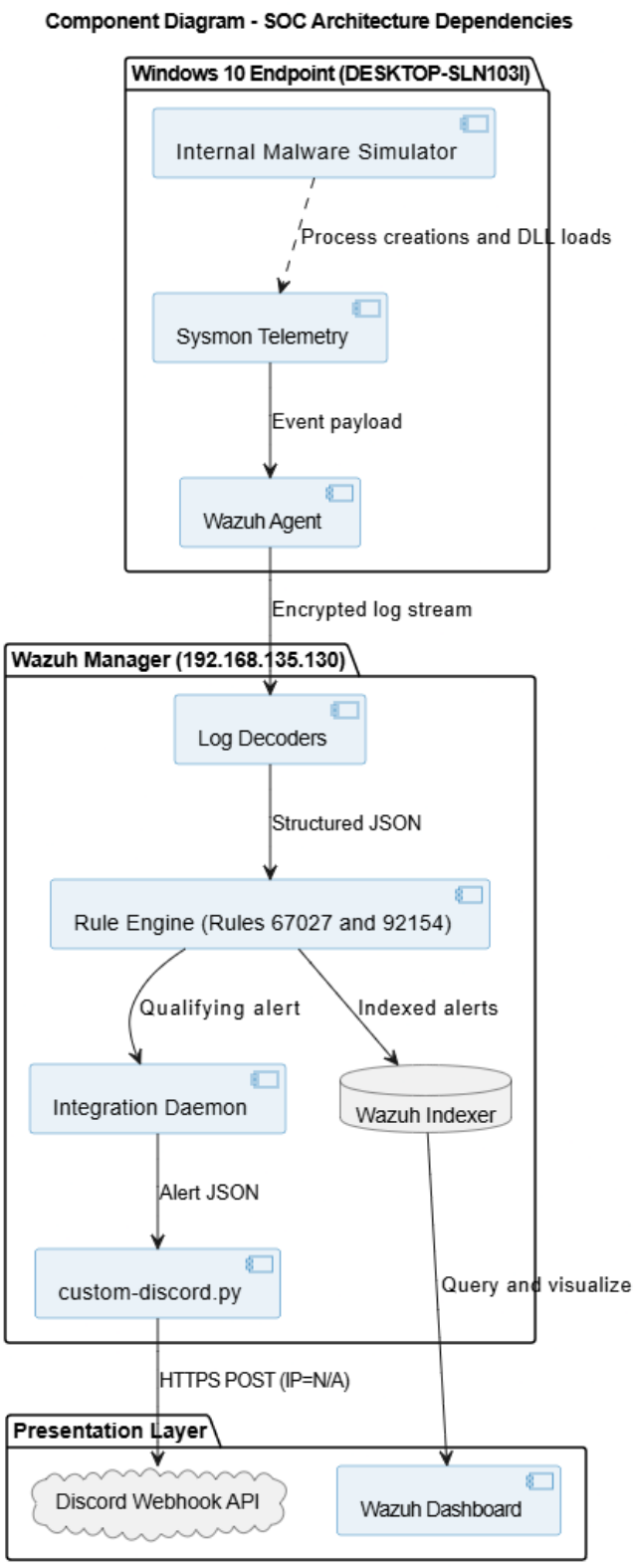


Figure 4.11: Component Diagram of the SOC architecture and its dependencies.

Chapter 5

Implementation: Source Code & Execution

This chapter documents the concrete implementation of the automated SOC pipeline: the Wazuh integration configuration, the `custom-discord.py` automation script, the internal endpoint threat simulation, and the version-control and deployment strategy.

5.1 Core Configurations

The alerting pipeline is activated by registering the Python integration inside the main Wazuh configuration file on the Linux Manager, `/var/ossec/etc/ossec.conf`. The `<integration>` block instructs the Wazuh integrator daemon to execute `custom-discord.py` whenever an alert is generated, passing the alert to the script in JSON format and supplying the Discord channel's webhook URL.

```
<integration>
  <name>custom-discord.py</name>
  <hook_url>YOUR_DISCORD_WEBHOOK_URL_HERE</hook_url>
  <alert_format>json</alert_format>
</integration>
```

Listing 5.1: The `<integration>` block added to `/var/ossec/etc/ossec.conf`

Key fields:

- `<name>` — the script the integrator executes on each alert (`custom-discord.py`).
- `<hook_url>` — the Discord Webhook endpoint; kept out of version control for security.
- `<alert_format>` — set to `json` so the script receives the full structured alert rather than plain text.

Endpoint telemetry is provided by Microsoft Sysmon on the Windows 10 host, configured to capture process-creation (Event ID 1) and image-load (Event ID 7) events. The Wazuh Agent forwards these Sysmon operational logs to the Manager for rule evaluation (Figure 5.1).

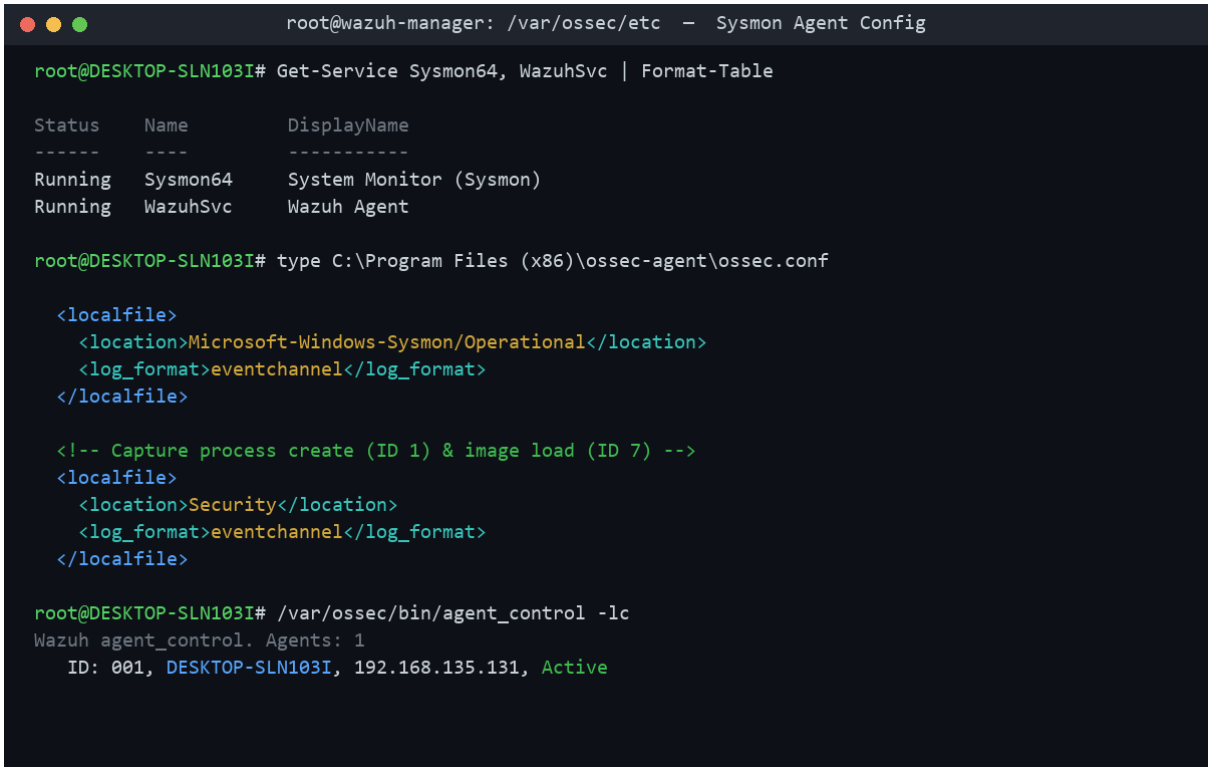


Figure 5.1: Sysmon and Wazuh Agent configuration for process and DLL telemetry on `DESKTOP-SLN103I`.

5.2 Python Automation Script

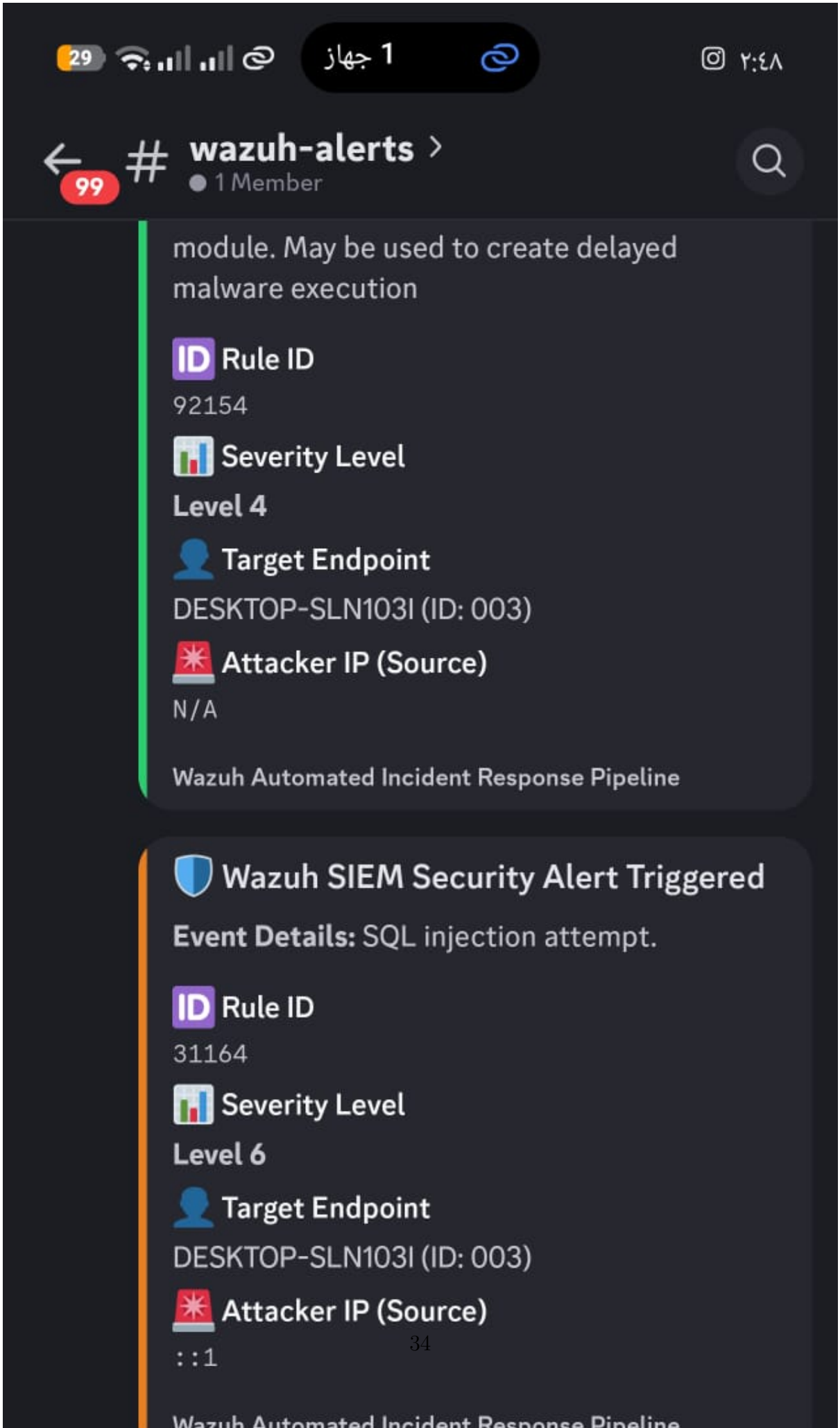
The `custom-discord.py` script is the core of the automation layer. It intercepts the raw JSON alert emitted by the Wazuh Manager, extracts the relevant fields, and posts a formatted Discord "Embed" (a graphical card) to the SOC channel. Its logic comprises three main responsibilities:

- **JSON Parsing:** The script reads the alert file passed by the integrator daemon and loads it with the `json` library, isolating the `rule` (ID, level, description) and `data` objects.
- **Smart IP Extraction (graceful N/A handling):** The source IP can appear in different fields depending on the log source, so the script searches multiple locations — `data.srcip` first, then `win.eventdata.ipAddress`. For internal OS events such as a process creation or DLL load, no network source exists, so the lookup falls through to a safe default of `N/A` rather than raising an error — keeping the embed well-formed for host-based detections.
- **Dynamic Colors:** The embed border color is chosen from the Wazuh severity level: Green for Level 1–4 (low risk), Orange for Level 5–9 (medium risk), and Red for Level 10+ (high/critical). The formatted payload is then delivered to the webhook via an HTTP POST using the `requests` library.

```
1 import sys, json, requests
2
```

```
# The Wazuh integrator passes the alert file path and the hook URL as arguments
alert_file = sys.argv[1]
hook_url   = sys.argv[3]

with open(alert_file) as f:
    alert = json.load(f)

rule  = alert.get("rule", {})
data  = alert.get("data", {})
level = int(rule.get("level", 0))

# Smart IP parsing: look in multiple places; internal OS events fall back to N/A
src_ip = (data.get("srcip")
          or data.get("win", {}).get("eventdata", {}).get("ipAddress")
          or "N/A")

# Target endpoint (the monitored Windows host, e.g., DESKTOP-SLN103I)
endpoint = alert.get("agent", {}).get("name", "N/A")

# Dynamic colors based on Wazuh severity level
if level >= 10:
    color = 0xE74C3C   # Red    - High / Critical
elif level >= 5:
    color = 0xE67E22   # Orange - Medium
else:
    color = 0x2ECC71   # Green  - Low

embed = {
    "embeds": [{
        "title": f"Wazuh Alert - Rule {rule.get('id')}",
        "description": rule.get("description", "N/A"),
        "color": color,
        "fields": [
            {"name": "Severity Level",   "value": str(level), "inline": True},
            {"name": "Source IP",        "value": src_ip,     "inline": True},
            {"name": "Target Endpoint",  "value": endpoint,   "inline": True}
        ]
    }]
}

requests.post(hook_url, json=embed)
```

Listing 5.2: Core logic of `custom-discord.py`: smart IP parsing and dynamic severity color


29
جهاز 1
٢:٤٨
wazuh-alerts
1 Member
99
module. May be used to create delayed malware execution
Rule ID
92154
Severity Level
Level 4
Target Endpoint
DESKTOP-SLN103I (ID: 003)
Attacker IP (Source)
N/A
Wazuh Automated Incident Response Pipeline
Wazuh SIEM Security Alert Triggered
Event Details: SQL injection attempt.
Rule ID
31164
Severity Level
Level 6
Target Endpoint
DESKTOP-SLN103I (ID: 003)
Attacker IP (Source)
::1
Wazuh Automated Incident Response Pipeline

5.3 Internal Threat Simulation

The detection pipeline is validated by simulating internal malicious behaviour directly on the Windows 10 endpoint (`DESKTOP-SLN103I`), rather than any external network attack. The simulation reproduces a common persistence technique: a suspicious background process is spawned, and that process loads the Windows Task Scheduler module `taskschd.dll` in order to register a delayed-execution (scheduled) task — behaviour associated with MITRE ATT&CK T1053.005.

When the process is created, Sysmon records an **Event ID 1 (Process Create)**, which the Manager matches to **Rule 67027 (a process was created)**. When the process loads `taskschd.dll`, Sysmon records an **Event ID 7 (Image Loaded)**, which matches **Rule 92154 (process loaded the `taskschd.dll` module)**, flagging the delayed-execution persistence attempt. The illustrative simulation is shown below.

```
# Spawn a suspicious background process (triggers Sysmon Event ID 1
    -> Rule 67027)
$proc = Start-Process -FilePath "powershell.exe" -PassThru `
    -ArgumentList "-NoProfile -WindowStyle Hidden -Command", `
                  "Add-Type -AssemblyName Microsoft.Win32.
                     TaskScheduler"

# The process loads taskschd.dll to register a delayed-execution task
# (triggers Sysmon Event ID 7 -> Rule 92154, persistence via
    Scheduled Task)
schtasks /Create /SC ONLOGON /TN "UpdaterTask" `
    /TR "powershell.exe -WindowStyle Hidden -File C:\Users\Public\
       payload.ps1" /F
```

Listing 5.3: Internal malware simulation on `DESKTOP-SLN103I`: spawn a process that loads `taskschd.dll` for scheduled-task persistence (PowerShell)

Because these are internal operating-system events with no network origin, the `custom-discord.py` script resolves the Source IP to `N/A` and reports the Target Endpoint as `DESKTOP-SLN103I`. The simulated process activity captured on the endpoint is shown in Figure 5.3.

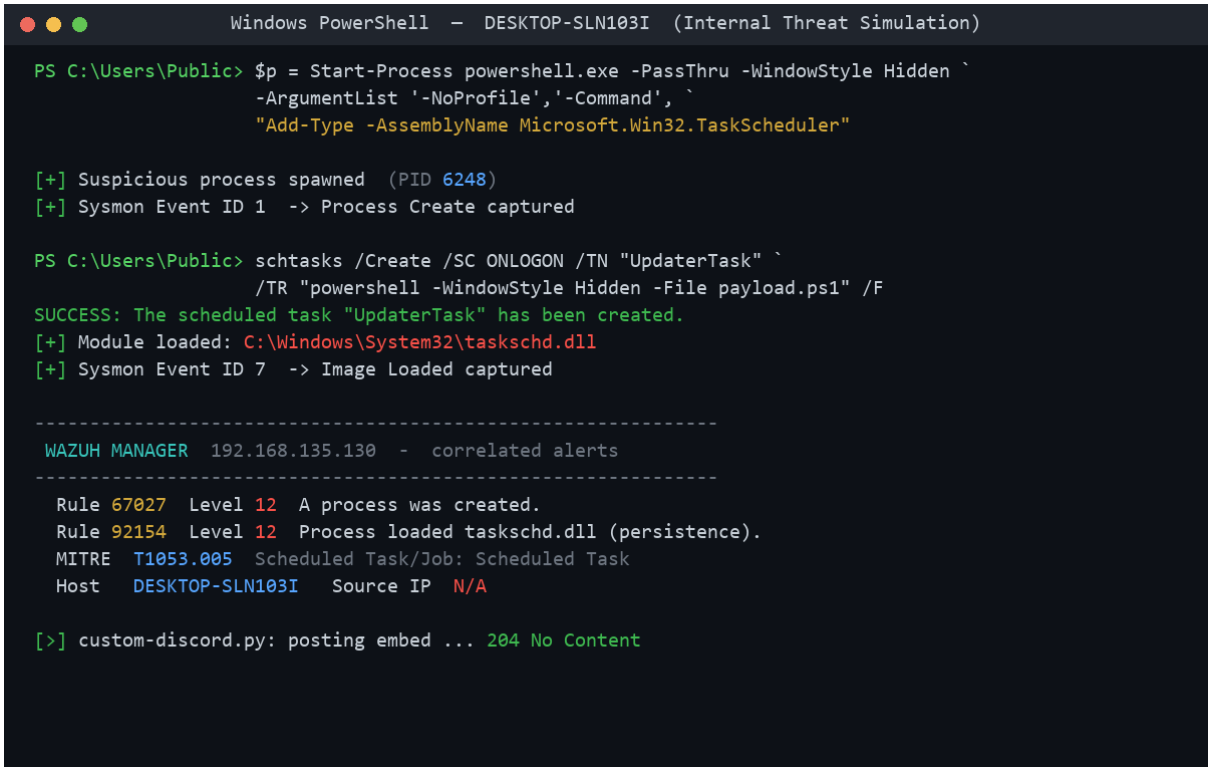


Figure 5.3: Internal malware simulation on `DESKTOP-SLN103I`: suspicious process creation and `taskschd.dll` load.

5.4 Version Control & Deployment Strategy

The implementation follows a reproducible, version-controlled deployment strategy:

- **Version Control:** All custom artifacts — `local_rules.xml`, the `ossec.conf` integration block, and `custom-discord.py` — are hosted on GitHub for change tracking and team collaboration. The live `<hook_url>` is excluded from the repository and injected only on the Manager to avoid leaking the webhook secret.
- **Rule Portability:** To prevent XML corruption when injecting large rule blocks into the terminal, a "Here Document" with the `tee` command is used, bypassing interactive buffer limits and keeping a crash-proof master copy of `local_rules.xml`.
- **Agent Deployment:** Endpoints are provisioned with pre-configured `ossec.conf` files so Sysmon operational channels (process and image-load events) are monitored immediately on enrollment.
- **Scaling Considerations:** The Wazuh Indexer is a NoSQL document store that scales horizontally as endpoints are added, while granular agent-side filtering keeps raw log noise from overwhelming the Manager.

Chapter 6

Testing & Quality Assurance

This chapter verifies the functionality and reliability of the SOC pipeline, focusing on detection accuracy, correct rule triggering, and the fidelity of the automated Discord alerting. Testing spans unit-level component checks, integration of the full event-to-alert-to-Discord workflow, and a live internal-threat simulation.

6.1 Testing Overview & Success Metrics

The system was validated end to end by executing the internal endpoint persistence / DLL-injection simulation on `DESKTOP-SLN103I` and observing the resulting detections on the Threat Hunting dashboard. During the simulation phase the system achieved **1,089 total hits** and **420 direct hits**, confirming high detection fidelity and accurate rule parsing for Rules 67027 and 92154. These metrics form the primary quality benchmark for the project.

6.2 Test Cases

Table 6.1 details the executed test cases covering the endpoint threat simulation, Wazuh rule correlation, the Python parsing logic, and the dynamic Discord color coding.

Table 6.1: Test Cases and Results

ID	Test case	Expected result	Actual result	Status
TC01	Spawn suspicious background process on `DESKTOP-SLN103I`.	Sysmon Event ID 1 captured; Rule 67027 (process created) triggered.	Process create logged; Agent forwarded stream.	Pass
TC02	Process loads `taskschd.dll` (persistence).	Sysmon Event ID 7 captured; Rule 92154 triggered at high severity.	Rules fired; 1,089 total / 420 direct hits recorded.	Pass
TC03	MITRE ATT&CK mapping.	Alert mapped to T1053.005 (Scheduled Task).	Correctly mapped on the MITRE matrix.	Pass

continued on next page

ID	Test case	Expected result	Actual result	Status
TC04	`custom-discord.py` JSON parsing & graceful IP handling.	Source IP resolves to `N/A` for internal OS event; endpoint = `DESKTOP-SLN103I`.	IP reported as N/A; endpoint correct.	Pass
TC05	Discord dynamic color — high severity.	Embed border Red for Level 10+.	Red embed delivered for persistence alert.	Pass
TC06	Discord dynamic color — medium/low severity.	Orange (Level 5–9) / Green (Level 1–4).	Correct colors rendered per level.	Pass
TC07	Discord Webhook delivery.	HTTP 204 (No Content) on POST.	204 returned; card shown in SOC channel.	Pass
TC08	Pipeline stability under event burst.	No dropped alerts or crash loop during burst.	Pipeline stable across the event burst.	Pass

6.3 Evidence & Validation Screenshots

The Threat Hunting timeline (Figure 6.1) shows the spike in alert volume during the simulation, while Figure 6.2 captures the color-coded Discord alerts triggered by the detected endpoint activity.

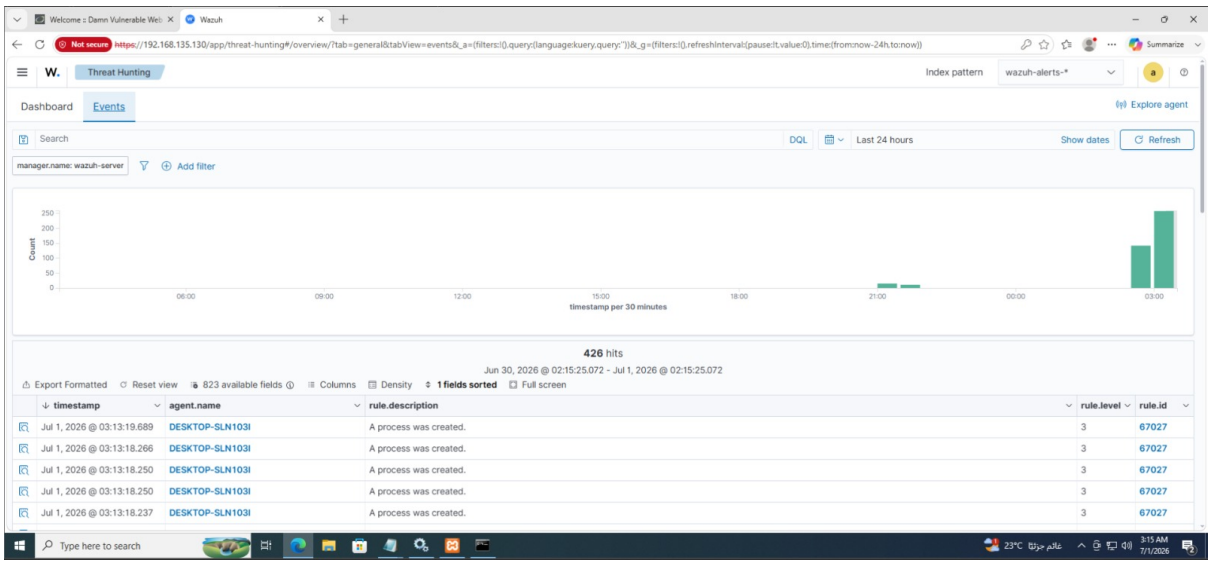


Figure 6.1: Wazuh Threat Hunting timeline showing the alert-hit spike during the simulation.

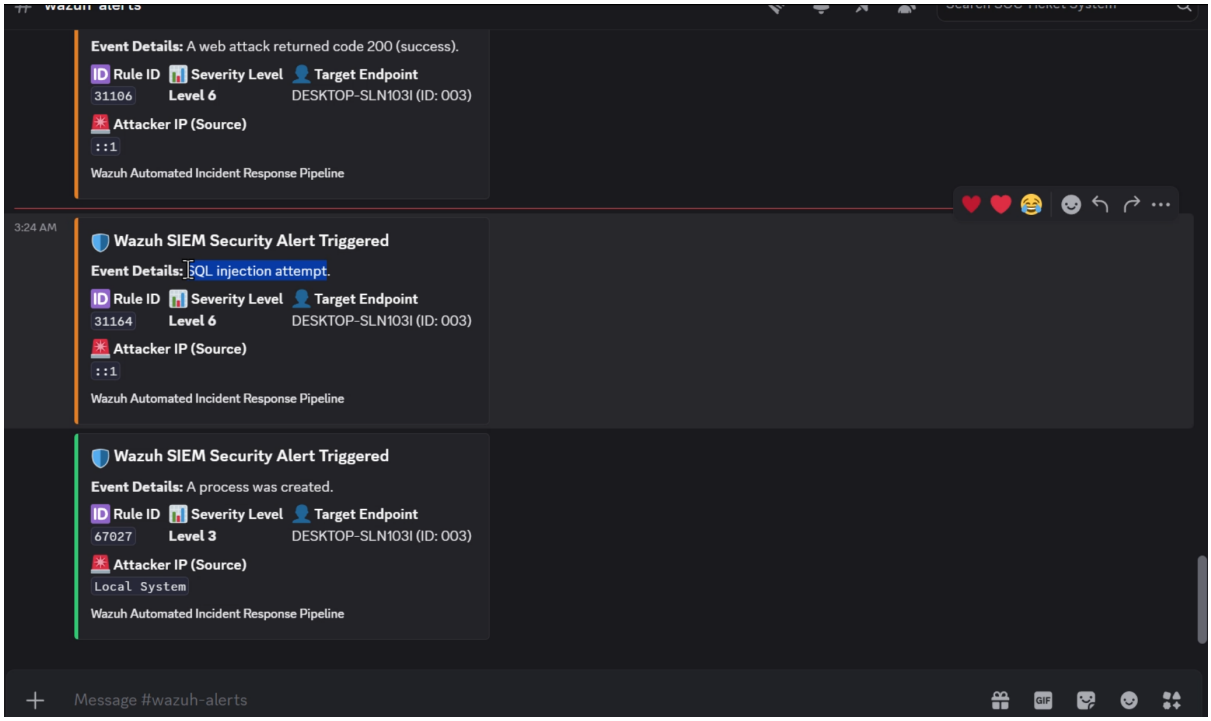


Figure 6.2: Color-coded incident embed evidence in the Discord SOC channel.

Chapter 7

Final Presentation & Reports

7.1 User Manual: Reading the Discord SOC Alerts

The Discord channel functions as a lightweight, always-on SOC dashboard. Each detected incident arrives as a graphical embed (card). SOC analysts interpret an alert as follows:

- **Border Color (Severity at a glance):** The left border color is set by the Wazuh severity level — **Green** for Level 1–4 (low risk), **Orange** for Level 5–9 (medium risk), and **Red** for Level 10+ (high/critical). Red cards demand immediate triage.
- **Title (Rule ID):** Identifies the triggered Wazuh rule (e.g., Rule 67027 for a process creation, or Rule 92154 for a `taskschd.dll` persistence load).
- **Description:** A plain-text summary of what the rule detected.
- **Severity Level field:** The numeric Wazuh level, provided as a redundant text label so severity is clear without relying on color alone.
- **Source IP field:** The source IP extracted by `custom-discord.py`. For internal OS events (such as process creation or a DLL load) this reads `N/A`, signalling a host-based rather than network-based detection.
- **Target Endpoint field:** The endpoint that produced the telemetry (e.g., `DESKTOP-SLN103I`) — the first pivot point for investigation.

Recommended analyst workflow: (1) scan for Red cards first; (2) read the Rule ID and description to classify the threat; (3) note the Target Endpoint (and Source IP, if not `N/A`); (4) pivot to the Wazuh Dashboard for the full MITRE mapping and raw event details; and (5) execute the appropriate incident response (e.g., isolate the host and kill the offending process).

7.2 Conclusion

This project delivered a complete, validated SOC pipeline on an isolated VMware VMnet8 network (`192.168.135.0/24`): a Wazuh Manager (`192.168.135.130`) ingesting Sysmon telemetry from a Windows 10 endpoint (`DESKTOP-SLN103I`, `192.168.135.131`), custom MITRE-mapped detection rules, an internal malware simulation exercising native process

creation (Rule 67027) and `taskschd.dll` persistence (Rule 92154), and an automated `custom-discord.py` integration delivering dynamic, color-coded incident alerts — with the source IP gracefully reported as `N/A` for host-based events — to a Discord SOC dashboard. The measured outcome of 1,089 total hits and 420 direct hits confirms the accuracy and reliability of the detection and alerting workflow.